Réseau Télémétrique de Mesures en Temps Réel.
Mesures Sismiques
R.T.M.T.R_1

Troisième revue
ETUDIANT 3

PLAN RTMTR 1

ANALYSE

VERS

CONCEPTION

Diapositive 4 : Expression du besoin
Diapositive 6 : Synoptique
Diapositive 9 : Diagramme de classe d’analyse
Diapositive 11 : Les contraintes matérielles et logicielles
Diapositive 12 : Structuration de la base de données
Diapositive 13 : Ihm test pour enregistrement
Diapositive 19 : Ihm recherche à partir de maintenant
Diapositive 24 : Ihm Recherche par période
Diapositive 30 : Conclusion

Présentation du projet

L’entreprise Vision Informatique souhaite réaliser un Réseau Télémétrique de Mesures en Temps Réel de stations connectées qui fourniront un flot de données sismiques et la position du lieu d’acquisition des mesures.

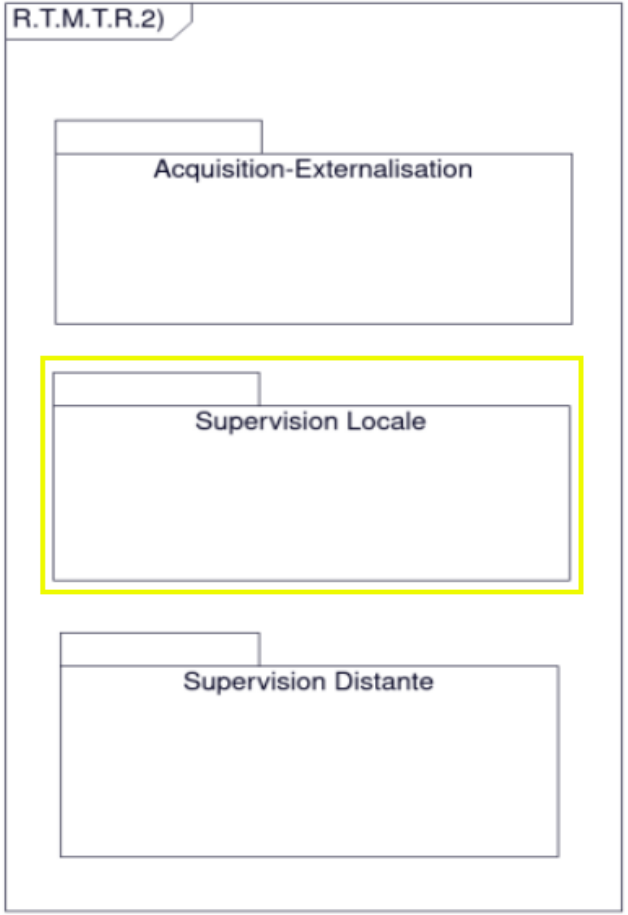
But :

- Fournir un flot de données sismiques et de position
- Présenter les données comme observatoire d'activités sismiques.
- Sensibiliser les élèves des collèges aux risques majeurs locaux tels que tsunamis, glissements de terrain, volcans, lahars et séismes.

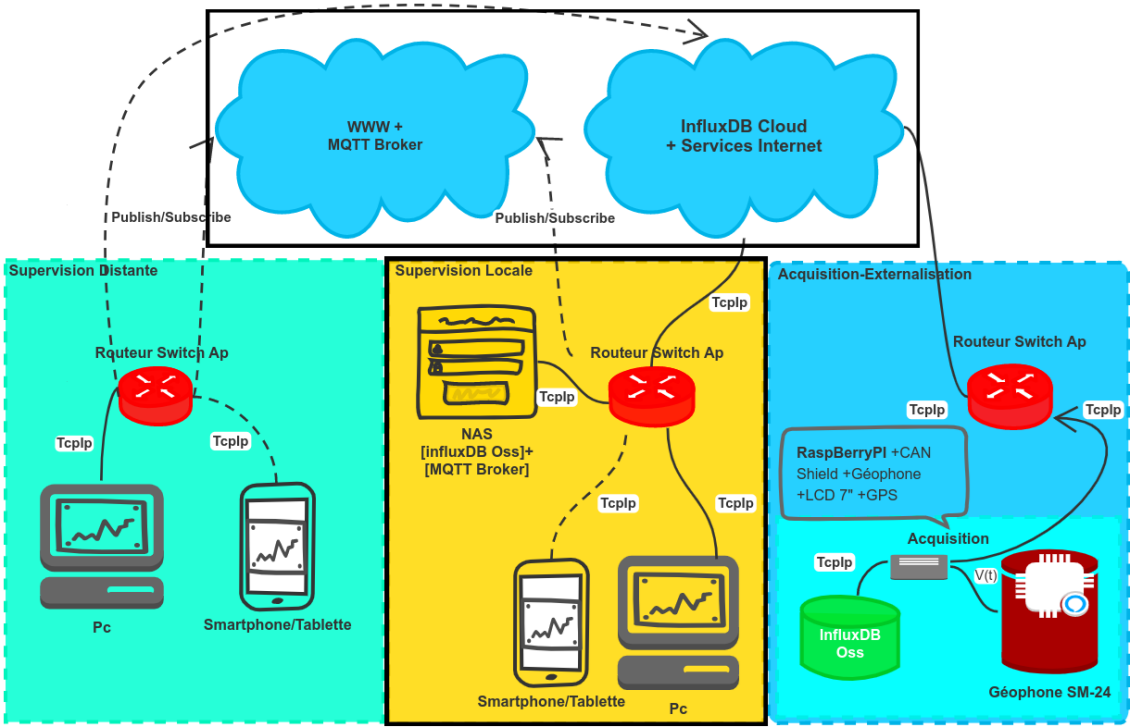
Spécification

Le système sera découpé en 3 sous-systèmes comme illustré ci-contre

J’ai été affecté à la partie Supervision Locale



Synoptique


WWW +
MQTT Broker
InfluxDB Cloud
+ Services Internet
Publish/Subscribe
Publish/Subscribe
Supervision Distante
Routeur Switch Ap
Tcplp
Tcplp
Pc
Smartphone/Tablette
Supervision Locale
Tcplp
Routeur Switch Ap
Tcplp
NAS
[influxDB Oss]+
[MQTT Broker]
Tcplp
Tcplp
Smartphone/Tablette
Pc
Acquisition-Externalisation
Routeur Switch Ap
Tcplp
Tcplp
RaspBerryPI +CAN
Shield +Géophone
+LCD 7" +GPS
Acquisition
Tcplp
V(t)
InfluxDB
Oss
Géophone SM-24

Expression du besoin

Objectifs :

- Lire les données enregistrées sur la base de données
- En extraire les données sur une période ou pendant une durée limitée.
- Esquisser un graphique à partir de ces données
- Envoyer ces données à la supervision distante via un broker mqtt

Les contraintes matérielles et logicielles

CHOIX MATÉRIELS

1 PC

1 NAS Synology

1 Switch

1 Broker Mqtt

1 Tablette Android

CHOIX LOGICIELS

Pour le PC : Linux, Framework Qt5

Pour le NAS : InfluxDb_Oss

Classe influxdb.hpp :

https://github.com/TheYonkk/influxdb-cpp-2

Diagramme des cas d'utilisation

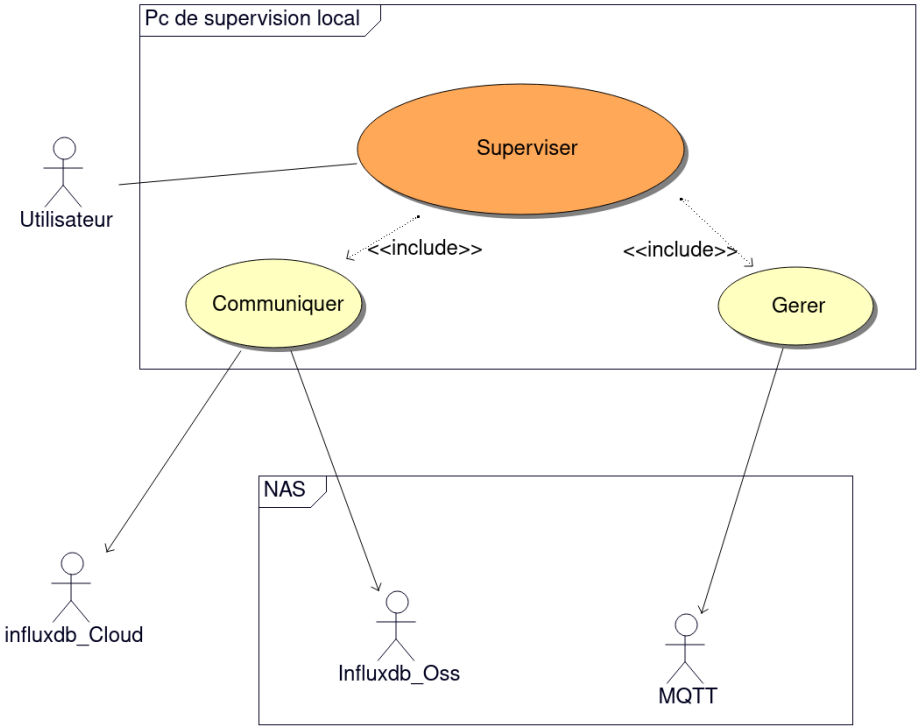


Diagramme de séquence

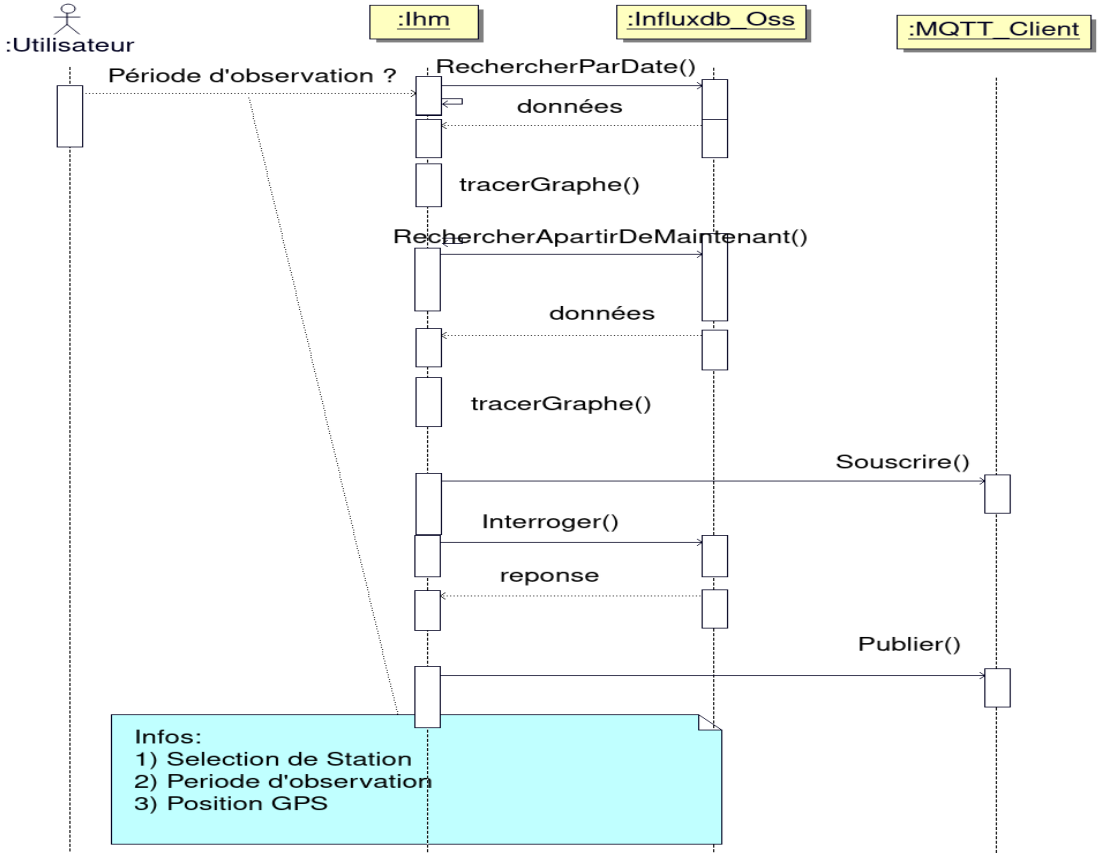


Diagramme de classe d'analyse

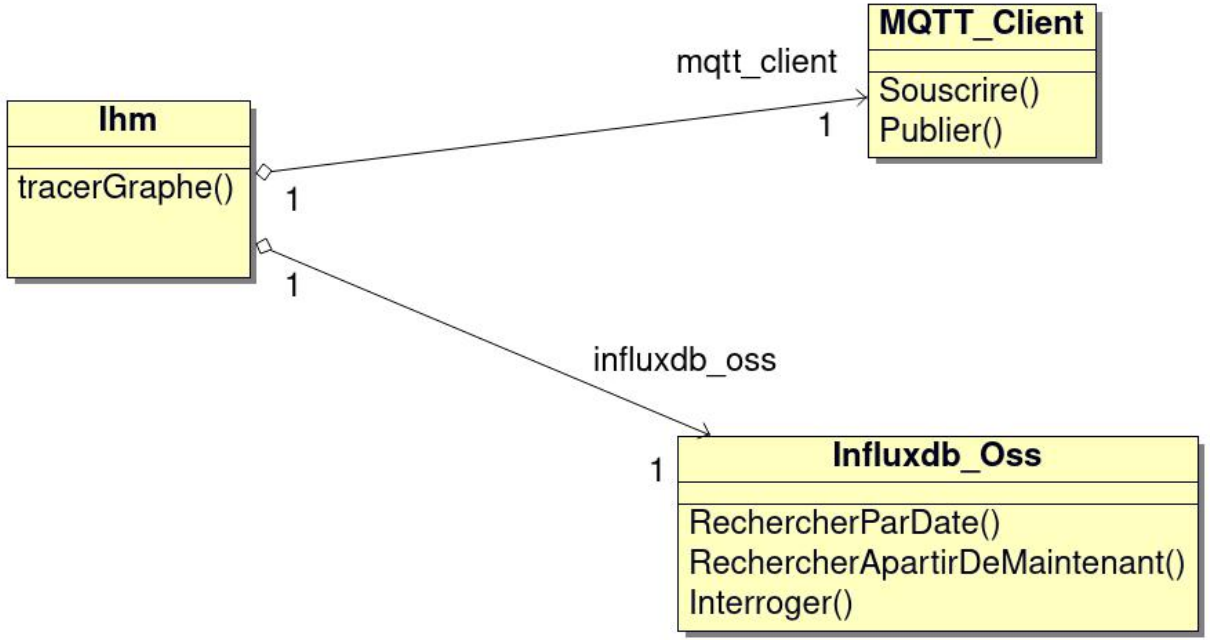
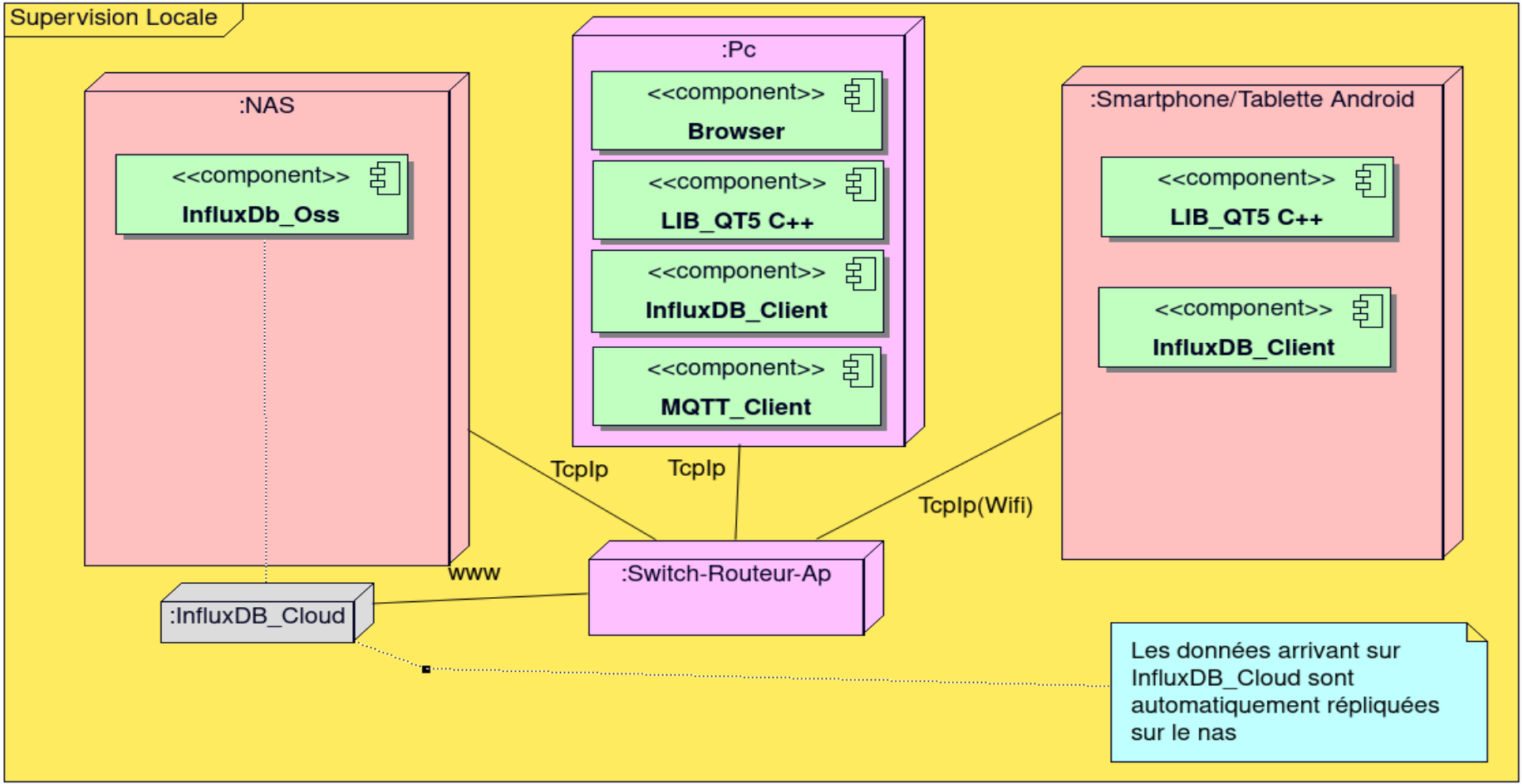


DIAGRAMME DE DÉPLOIEMENT



Structuration de la base de données

L'équipe a décidé de structurer les bases de données de la façon suivante :

- Structure de la base de données

  bucket -> measurement -> field

  InfluxDB_Oss définit trois zones

  1. bucket : choix du nom de la base de données
  2. measurement : choix du nom du measurement
  3. field : choix des noms des mesures effectives

- Conception de la base de données

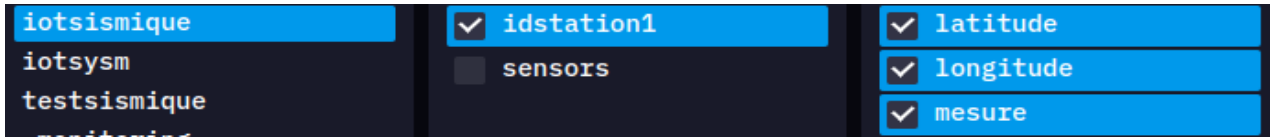
  bucket -> measurement -> field

  iotsismique -> idstation1 -> mesure

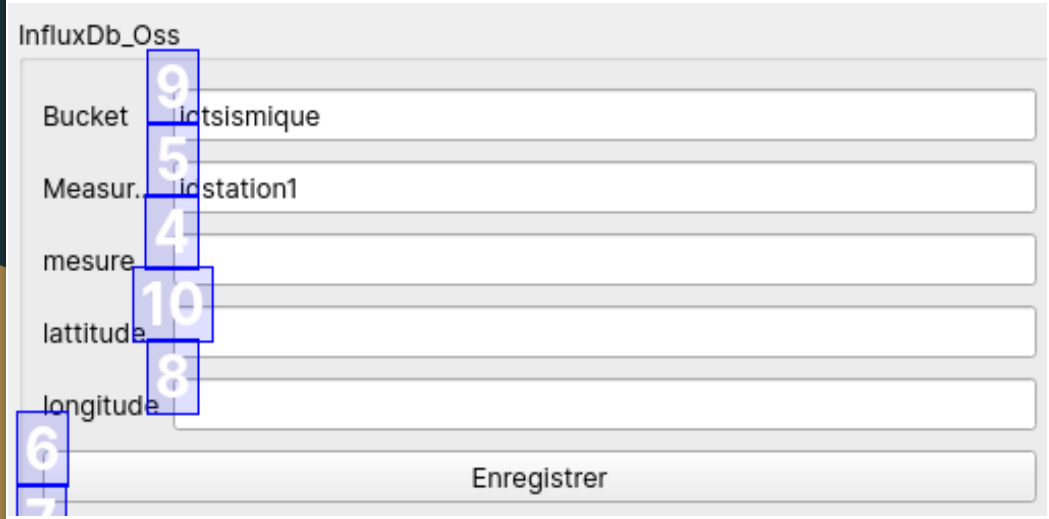
  latitude

  longitude

- Structure résultante dans InfluxDB_Oss



IHM TEST POUR ENREGISTREMENT



9. Nom du bucket de la base de données
5. Nom du measurement de la base de données
4. Nom du premier field de la base de données
10. Nom du deuxième field de la base de données
8.Nom du troisième field de la base de données
6. Bouton d'enregistrement dans la base de données

IHM : Extrait de code de l'enregistrement

```
void Widget::on_enregistrer_clicked()
{
        QString IdStation = ui->measurement->text();
        int lamesure      = ui->mesure->text().toInt();
        int lalattitude   = ui->lattitude->text().toInt();
        int lalongitude   = ui->longitude->text().toInt();

    influxdb_Oss::server_info si("192.168.1.200"  ///connexion
            ,49153, "btssnir"
        jeton masque
            ,"iotsismique");

    influxdb_Oss::builder()              ///Requete d'enregistrement
            .meas(IdStation.toStdString())
            .field("mesures", lamesure)
            .field("latitude", lalattitude)
            .field("longitude",lalongitude)
            .post_http(si);
}
```

J'ai créé une fonction me permettant d'enregistrer des données sur la base de données pour pouvoir les exploiter par la suite.

Comment faire des requêtes sur la BD avec l'outil InfluxDB.

Exemple de requête simple: avec l'outil intégré fourni par influx

```
from(bucket: "iotsismique")
  |> range(start: -3m)
 |> filter(fn: (r) => r._measurement == "idstation1"
  |> filter(fn: (r) => r._field == "mesure")
```

Comment faire des requêtes sur la BD avec Qt5 C++ (pour mon application).

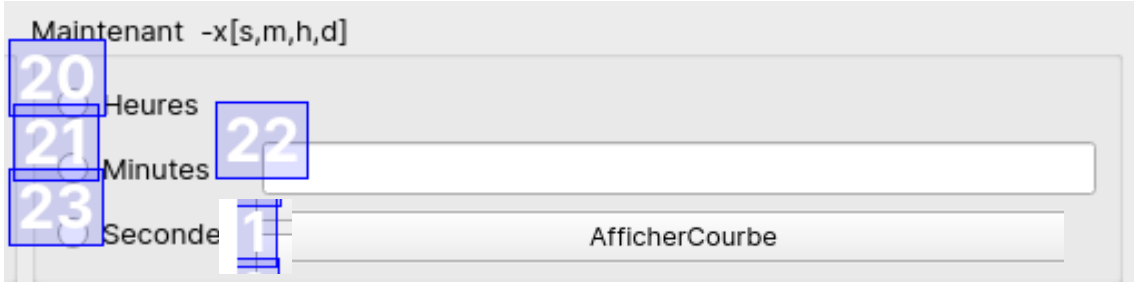
```
string query("from(bucket: \\\"iotsismique\\\")
|> range(start: -3h)
|>filter(fn: (r)=>r[\\\"_measurement\\\"] == \\\"idstation1\\\")
|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\"mesure\\\")");
flux_query(resp, query, *servdb_);
qDebug() << resp.c_str();
```

Résultat de la requête

1 ,result,table,_start,_stop,_time,_value,_field,_measurement
2 ,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:41:58.199426132Z,18.86,**mesure**,**idstation1**
3 ,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:41:59.200188759Z,49.2,**mesure**,**idstation1**
4 ,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:42:00.200046013Z,11.49,**mesure**,**idstation1**

Les numéros de lignes ont été ajoutés volontairement. La ligne **n°1** montre l'organisation d'une réponse à une requête. Le traitement commencera à partir de la deuxième ligne.

IHM Test:Recherche de maintenant à t-1 (t (s,m,h,day)



Cette partie de l'IHM est utilisée pour rechercher à partir de maintenant les données existant sur la base pendant une durée renseignée

1.Bouton permettant la recherche

20. Bouton radio pour l'unité de temps (en heures)

21. Bouton radio pour l'unité de temps (en minutes)

22. Entrée de la durée de recherche

23. Bouton radio pour l'unité de temps (en secondes)

Extrait de code de l'IHM : Connexion BDD

```
servdb_=new server_info ("192.168.1.200",
49153,
"btssnir",
jeton masque
"iotsismique");
```

Afin de pouvoir rechercher dans la base de données, il est nécessaire de s'y connecter.

Exécution de la requête

```
string resp;
string query = s1+sbucket+range1+filter1+filter2;
qDebug() << query.c_str();
flux_query(resp, query, *servdb_);
```

Exécution requête

Constitution de la requête

```
string query = s1+sbucket+range1+filter1+filter2;
```

=

Reconstitution en programme de:

```
string query("from(bucket:
\\\"iotsismique\\\")

|> range(start: -3h)

|>filter(fn: (r)=>r[\\\"_measurement\\\"] ==
\\\"idstation1\\\")

|>filter(fn: (r)=>r[\\\"_field\\\"] ==
\\\"mesure\\\")");
```

Exemple de résultat

```
,result,table,_start,_stop,_time,_value,_field,_measurement
,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:41:58.199426132Z,18.86,mesure,idstation1
,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:41:59.200188759Z,49.2,mesure,idstation1
,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:42:00.200046013Z,11.49,mesure,idstation1
```

,_result,0,2022-11-27T10:41:58.017915238Z,2022-11-27T10:42:58.017915238Z,2022-11-27T10:42:00.200046013Z,11.49,mesure,idstation1

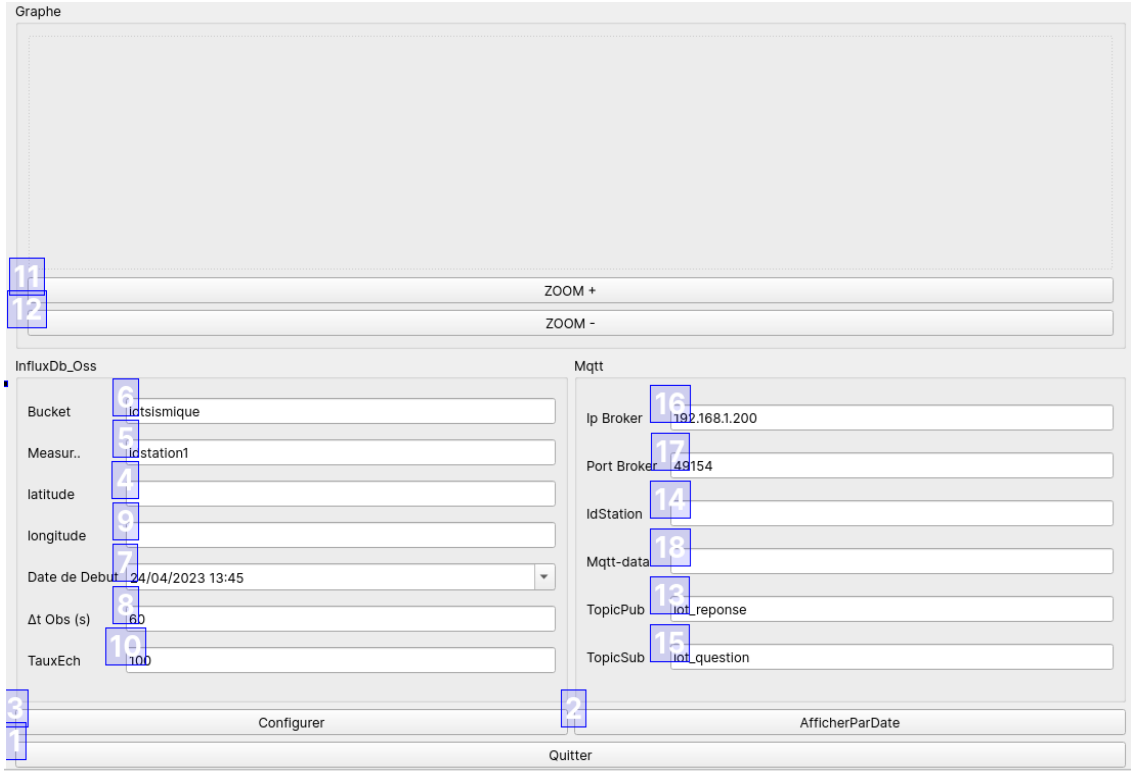
```
void Widget::ExtraireValueDate(QList<QByteArray>liste,unsigned int nbech)
 {
     for (unsigned int i=0; i<nbech;i++)
     {
     spltVirgule_ = liste[i].split(',');   //  Sépare la trame par champs avec une virgule comme moyen d'opération
     QByteArray Date = spltVirgule_[5];
     QByteArray Valeur = spltVirgule_[6];
```

IHM Recherche par période (DateDebut - DateFin)



Cette partie de l'IHM est utilisée pour rechercher durant une période définie les données existantes dans la base de données.

Description de l'IHM



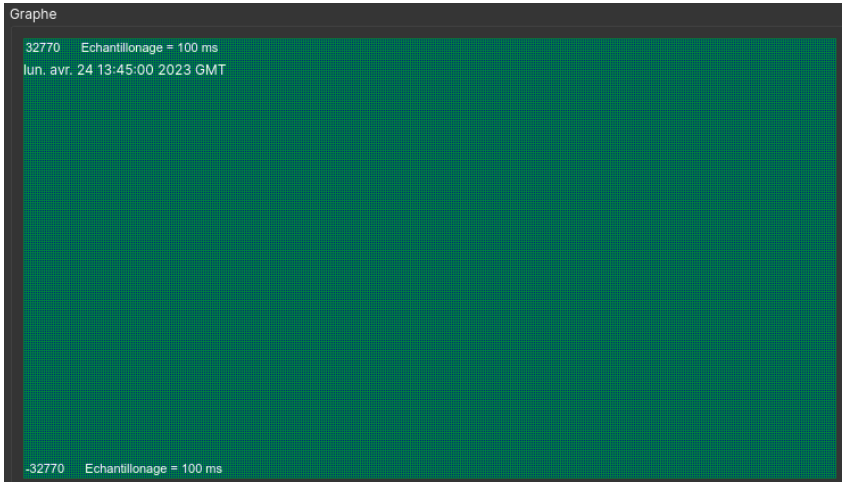
1. Bouton pour fermer l'IHM
2. Bouton permettant l'affichage de la courbe
3. Bouton permettant de revenir sur cette interface après avoir affiché la courbe
4. Champ d'enregistrement de latitude
5. Champ de measurement de la base
6. Nom de la base
7. Date de la période d'observation
8. Renseignement de la durée d'observation
9. Champ de longitude
10. Taux d'échantillonnage
11. Bouton de zoom avant du graphique
12. Bouton de zoom arrière du graphique
13. Topic de publication MQTT
14. Id de la station
15. Topic de souscription MQTT
16. Adresse ip du broker MQTT
17. Port du broker MQTT
18. Récupération de la donnée envoyée (date période)

Affichage graphique des données lues

Configuration

Commencer par initialiser le fond du graphique en renseignant :

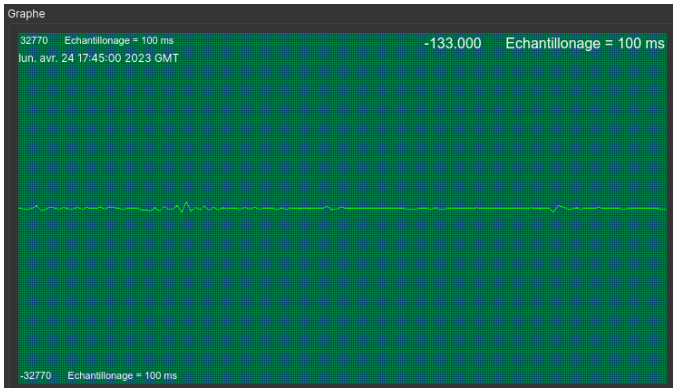
- la taille d’un point,
- la taille de l'échelle,
- la couleur de fond.



Puis, grâce à une méthode, utiliser les données lues pour tracer un graphique

Pour ce faire, on utilise la méthode TraceGraphe qui fait une boucle qui rajoute un point grâce à la liste de valeurs.

```
ExtraireValueDate(spltCrLf_,nbEchantillons);
 ui->plot_4->setUnits("     Echantillonage = "+ui->ltauxech->text()+" ms");
TraceGraphe( ListHeure, ListValeur);
```

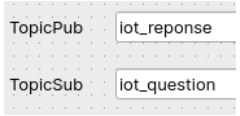


Broker MQTT pour supervision distante

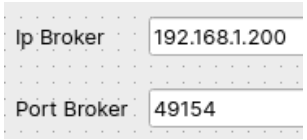
Afin de réaliser cette dernière partie, il faut créer toute la partie de connexion sur un broker mqtt avec un topic question afin d'y récupérer une valeur qui est une date d'observation qui sera ensuite envoyée vers mon programme. Puis, il se chargera d'envoyer sur un topic réponse les données correspondantes à cette date d'observation.

Pour ce faire, il nous faut créer une connexion mqtt sur l'IHM avec :

les topics de publication et de souscription



N'oublions pas l'adresse ip et le port permettant l'échange entre les deux services



ainsi qu'une connection grâce à un objet mqtt qui utilise la fonction de connexion de la classe

```
mqtt_sub->Connexion(ui->lipbroker->text(),ui->lportbroker->text().toInt(),ui->ltopicsub->text());
mqtt_pub->Connexion(ui->lipbroker->text(),ui->lportbroker->text().toInt(),ui->ltopicpub->text());
connect(mqtt_sub,SIGNAL(onReceived(QByteArray)),this,SLOT(RecevoirMessage(QByteArray)));
```

Une fois la connexion effectuée, il faut créer une fonction faisant le même traitement de données des trames afin d'envoyer les résultats sur le topic réponse pour la supervision distante de manière automatique.

L'envoie des résultats se produit grâce à la fonction Publish de la classe

```
mqtt_pub->Publish(6,ui->ltopicpub->text(),(QString::number(ListValeur.first())).toUtf8());
```

Adaptation du programme

Ce projet à été pensé pour être utilisable sur tablette en la branchant via câble usb sur l'ordinateur utilisant ce programme.

Pour qu'elle soit compatible, il faut rentrer une commande, montrée ci-dessous, afin de reconnaître l'appareil.

Ne pas oublier d'accepter le débogage USB pour que la liaison fonctionne

```
e11g1@Poste11:~$ adb devices
List of devices attached
adb server version (41) doesn't match this client (39); killing...
* daemon started successfully
G4NKFPC022002SY device

e11g1@Poste11:~$
```

Conclusion

Ce projet nous a permis de découvrir différentes façons d’accéder à une base de données pour y lire les données et les utiliser dans plusieurs activités variées..

Au niveau de la gestion du projet en équipe, nous avons réussi à bien nous répartir les tâches afin de réaliser nos objectifs dans les temps et l'ambiance générale du groupe était très bonne.